

```
localStorage.setItem("tema", "escuro");
```

A API Web Storage

Como guardar dados direto no navegador — a alternativa moderna e mais eficiente aos cookies.

■ OBJETIVO

O que você vai saber fazer ao final

01

Entender

o que é a API Web Storage e por que ela existe.

02

Utilizar

os métodos de `localStorage` e `sessionStorage`.

03

Escolher

o tipo certo de armazenamento para cada caso.

Por onde vamos passar

01 O que é a API Web Storage

03 Objetos, eventos, limites e segurança

05 Referências bibliográficas

02 localStorage e sessionStorage na prática

04 Exercício de fixação

✦ Leitura recomendada: MDN Web Docs

01

— O que é Web Storage

Um "armário" de dados dentro do próprio navegador.

■ DEFINIÇÃO

Armazenar dados no navegador do usuário

Um mecanismo do lado cliente para guardar informação — uma [alternativa mais moderna e eficiente aos cookies](#).

A ESSÊNCIA Guarda pares **chave** → **valor**, presos à origem (domínio) do site.

Quatro vantagens sobre os cookies

01 **Maior capacidade**

Muito mais espaço que os ~4 KB dos cookies.

02 **Dados persistentes**

O localStorage sobrevive a fechar e reabrir o navegador.

03 **Facilidade de uso**

API simples e intuitiva para guardar e recuperar.

04 **Melhora o desempenho**

Reduz requisições ao servidor — aplicação mais rápida.

Quando cada um faz sentido

	COOKIE	WEB STORAGE
Capacidade	~4 KB	~5–10 MB
Vai ao servidor?	Sim, em toda requisição	Nunca — só no cliente
Acesso no JS	<code>document.cookie</code>	API de métodos
Validade	Expires / Max-Age	Permanente ou por sessão

Mesma API, tempos de vida diferentes

`localStorage`

Persiste indefinidamente

Fica até ser removido manualmente ou pelo navegador.
Sobrevive a fechar e reabrir.

`sessionStorage`

Dura só a sessão atual

Existe enquanto a aba está aberta. Ao fechar, os dados são apagados.

02

— Na prática

Os métodos e um contador de verdade.

■ COMO FUNCIONA

Os métodos da API

`setItem(chave, valor)`

Armazena um par chave-valor.

`getItem(chave)`

Recupera o valor associado a uma chave.

`removeItem(chave)`

Remove um item específico.

`clear()`

Remove **todos** os itens.

`length · key(i)`

Quantos itens há, e o nome da i-ésima chave.

Ler ao carregar e incrementar

```
// lê o valor salvo, ou usa 0 como padrão
let contador = localStorage.getItem("contador") || 0;
document.getElementById("contador").textContent = contador;
document.getElementById("btnIncrementar")
  .addEventListener("click", function () {
    contador++;
    localStorage.setItem("contador", contador);
    document.getElementById("contador").textContent = contador;
  });
```

Zerar e remover do armazenamento

```
document.getElementById("btnZerar")
  .addEventListener("click", function () {
    contador = 0;
    localStorage.removeItem("contador");
    document.getElementById("contador").textContent = contador;
  });
```

SIMETRIA setItem grava · removeItem apaga um · clear apaga tudo.

■ A API É IDÊNTICA

O mesmo código, trocando uma palavra

```
let contador = sessionStorage.getItem("contador") || 0;  
sessionStorage.setItem("contador", contador);
```

Com `sessionStorage`, fechar a aba zera o contador. Com `localStorage`, ele é lembrado. **Uma palavra decide o tempo de vida.**

■ QUAL DELES USAR?

Uma pergunta decide

"Esse dado precisa **sobreviver ao fechar o navegador**?"

NÃO → `sessionStorage`

Dados temporários — o estado de um formulário em preenchimento, um rascunho.

SIM → `localStorage`

Dados persistentes — preferências do usuário, como tema e idioma.

03

— Indo além

Objetos, eventos, limites e segurança.

■ PEGADINHA • SÓ EXISTE TEXTO

O Storage só guarda **strings**

Para guardar objetos e arrays, serialize com JSON: stringify ao gravar, parse ao ler.

```
const usuario = { nome: "Ana", tema: "escuro" };  
// gravar: objeto → texto JSON  
localStorage.setItem("usuario", JSON.stringify(usuario));  
// ler: texto JSON → objeto  
const salvo = JSON.parse(localStorage.getItem("usuario"));  
console.log(salvo.tema); // "escuro"
```

O evento `storage`

Quando o `localStorage` muda, o navegador avisa as **outras abas** do mesmo site — ótimo para mantê-las em sincronia.

```
window.addEventListener("storage", function (e) {  
  console.log("Mudou a chave:", e.key);  
  console.log("Novo valor:", e.newValue);  
});  
  
// dispara nas OUTRAS abas, não na que alterou
```

■ DOIS CUIDADOS SÉRIOS

Limites e segurança

LIMITE DE ESPAÇO

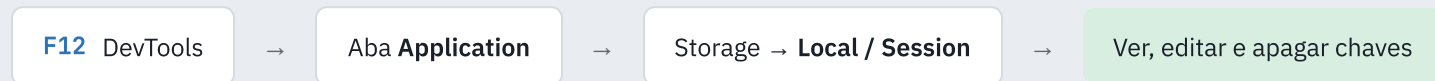
São ~5–10 MB, não infinito. Estourar a cota lança `QuotaExceededError` — proteja com `try/catch`.

⚠ NUNCA GUARDE SEGREDOS

Qualquer script da página lê o Storage (risco de **XSS**). Nada de senhas, tokens ou dados de cartão.

■ DICA DE DEPURAÇÃO

Veja e edite tudo no DevTools



04

— Exercício de fixação

A "Cápsula do Tempo Digital" — feita por humanos, para humanos. 

Cápsula do Tempo Digital

Construa uma página onde a pessoa escreve **mensagens para o seu "eu do futuro"**. As mensagens ficam guardadas no `localStorage` e reaparecem nas próximas visitas. O **rascunho em digitação** fica no `sessionStorage` — para você *sentir* a diferença entre os dois.

`localStorage`

Mensagens salvas — **lembradas** para sempre.

`sessionStorage`

Rascunho — some ao **fechar a aba**.

Do essencial ao caprichado

- 1 Campo de texto + botão que **grava a mensagem** no localStorage (guarde um array com JSON).
- 2 Ao **carregar a página**, listar todas as mensagens já guardadas.
- 3 Um botão para **apagar** uma mensagem (atualize o array e regrave).
- 4 Salvar o **rascunho** no sessionStorage a cada tecla (evento `input`) e restaurá-lo ao recarregar.
- 5 Mostrar um **contador de visitas** persistente no localStorage.

■ EXERCÍCIO 🛡️

Este exercício é à prova de atalhos

Use IA para *estudar* — mas a entrega tem que ser sua. Há três verificações combinadas:

1

Palavra-senha da aula

Guarde na chave `chaveDaAula` a palavra que o professor der **hoje**, ao vivo. A IA não conhece.

2

Defesa

Explique ao vivo **por que** o rascunho usa `sessionStorage` e as mensagens usam `localStorage`.

05

— Para aprofundar

Referências e a leitura recomendada.

Para aprofundar

★ LEITURA RECOMENDADA

MDN Web Docs — Web Storage API

A referência oficial e mais completa · developer.mozilla.org/pt-BR/docs/Web/API/Web_Storage_API

W3Schools. Disponível em: <http://www.w3schools.com>

MDN Web Docs — *Window.localStorage* & *Window.sessionStorage*. Disponível em: developer.mozilla.org